

salesdata.R

creation

2025-07-19

```
# 1. Load the libraries for this session
library(tidyverse)    # A collection of essential packages for data manipulation (dplyr), visualization (ggplot2), and more.
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr   1.5.1
## ✓ ggplot2     3.5.2      ✓ tibble     3.3.0
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr       1.1.0
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(janitor)      # Excellent for cleaning column names and other basic data cleaning tasks.
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##   chisq.test, fisher.test
```

```
library(lubridate)    # Makes working with dates and times much easier.
library(DataExplorer) # Great for automated exploratory data analysis and visualizing missing data.
library(randomForest) # A popular and powerful machine learning algorithm for prediction.
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
```

```
library(caret)           # For splitting data into training and testing sets
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(xgboost)         # The core package for the XGBoost algorithm
```

```
##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:dplyr':
##
##   slice
```

```
library(dplyr)
library(doParallel)      # Library for parallel processing
```

```
## Loading required package: foreach
##
## Attaching package: 'foreach'
##
## The following objects are masked from 'package:purrr':
##
##   accumulate, when
##
## Loading required package: iterators
## Loading required package: parallel
```

```
library (RcppRoll)
```

```
# 2. Import Data
```

```
sales_data<-read.csv("/home/creation/Project/Sales Analysis Project/sales_data/train.csv")
```

```
# 3. Display the first few rows of the original data
```

```
head(sales_data)
```

```
##           ID Store_id Store_Type Location_Type Region_Code      Date Holiday
## 1 T1000001         1         S1           L3           R1 01/01/2018         1
## 2 T1000002        253         S4           L2           R1 01/01/2018         1
## 3 T1000003        252         S3           L2           R1 01/01/2018         1
## 4 T1000004        251         S2           L3           R1 01/01/2018         1
## 5 T1000005        250         S2           L3           R4 01/01/2018         1
## 6 T1000006        249         S1           L3           R2 01/01/2018         1
##   Discount X.Order      Sales
## 1      Yes         9  7011.84
## 2      Yes        60 51789.12
## 3      Yes        42 36868.20
## 4      Yes        23 19715.16
## 5      Yes        62 45614.52
## 6      Yes        39 34211.22
```

```
# 4. Data Preparation & Cleaning
```

```
## Create a copy to work on, preserving the original import
```

```
df_clean <- sales_data
```

```
##Changing Column Name to more understandable Name
```

```
df_clean<-df_clean %>%
```

```
  rename(No_of_Orders=X.Order)
```

```
## Renaming Columns and Cleaning Whitespace
```

```
# Why: Standardizing column names makes them easier to type and reference.
```

```
df_clean<-df_clean %>%
```

```
  clean_names()
```

```
## Checking and Correcting Data Structure & Types
```

```
# Why: R needs to know if a column is a number, text, date, or category (factor) to perform
```

```
# the correct operations and create appropriate plots.
```

```
glimpse(df_clean)
```

```
## Rows: 188,340
## Columns: 10
## $ id          <chr> "T1000001", "T1000002", "T1000003", "T1000004", "T100000...
## $ store_id    <int> 1, 253, 252, 251, 250, 249, 248, 247, 246, 254, 245, 11,...
## $ store_type  <chr> "S1", "S4", "S3", "S2", "S2", "S1", "S1", "S1", "S3", "S...
## $ location_type <chr> "L3", "L2", "L2", "L3", "L3", "L3", "L1", "L1", "L1", "L...
## $ region_code <chr> "R1", "R1", "R1", "R1", "R4", "R2", "R2", "R3", "R3", "R...
## $ date        <chr> "01/01/2018", "01/01/2018", "01/01/2018", "01/01/2018", ...
## $ holiday     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
## $ discount    <chr> "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", ...
## $ no_of_orders <int> 9, 60, 42, 23, 62, 39, 40, 64, 62, 87, 36, 69, 69, 51, 5...
## $ sales       <dbl> 7011.84, 51789.12, 36868.20, 19715.16, 45614.52, 34211.2...
```

```
## Convert data types appropriately
# Why: `id`, `store_type`, etc., are categorical. `date` needs to be a Date object.
# `holiday` and `discount` are also categorical.
df_clean=df_clean %>%
  mutate(
    store_id=as.factor(store_id),
    store_type=as.factor(store_type),
    location_type=as.factor(location_type),
    region_code=as.factor(region_code),
    holiday=as.factor(ifelse(holiday==1,"Yes","No")),
    discount=as.factor(discount),
    date=dmy(date),
    sales=as.numeric(sales),
    no_of_orders=as.numeric(no_of_orders),
    date=as.Date(date,format="%d/%m/%y"),
  )

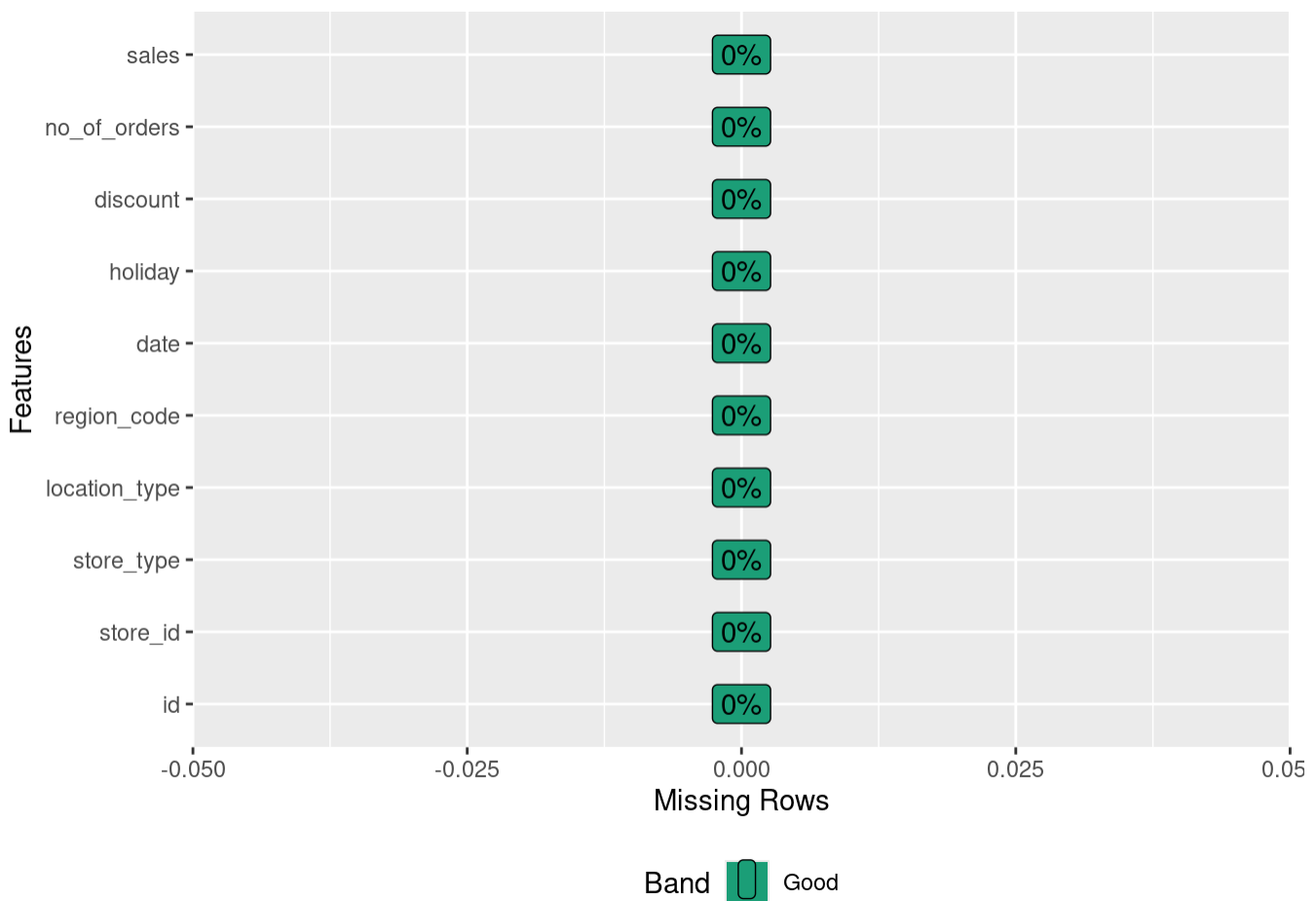
## Rechecking corrected data structures
glimpse(df_clean)
```

```
## Rows: 188,340
## Columns: 10
## $ id          <chr> "T1000001", "T1000002", "T1000003", "T1000004", "T100000...
## $ store_id    <fct> 1, 253, 252, 251, 250, 249, 248, 247, 246, 254, 245, 11,...
## $ store_type  <fct> S1, S4, S3, S2, S2, S1, S1, S1, S3, S4, S4, S4, S4, S4, ...
## $ location_type <fct> L3, L2, L2, L3, L3, L3, L1, L1, L1, L1, L1, L2, L2, L1, ...
## $ region_code <fct> R1, R1, R1, R1, R4, R2, R2, R3, R3, R1, R2, R1, R2, R1, ...
## $ date        <date> 2018-01-01, 2018-01-01, 2018-01-01, 2018-01-01, 2018-01...
## $ holiday     <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Y...
## $ discount    <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Y...
## $ no_of_orders <dbl> 9, 60, 42, 23, 62, 39, 40, 64, 62, 87, 36, 69, 69, 51, 5...
## $ sales       <dbl> 7011.84, 51789.12, 36868.20, 19715.16, 45614.52, 34211.2...
```

```
## Check for Missing Values
# Why: Missing values (NA) can break calculations and models. We need to identify the
# and decide on a strategy (remove, impute, etc.).
colSums(is.na(df_clean))
```

```
##          id      store_id  store_type location_type  region_code
##          0          0          0          0          0
##      date      holiday      discount  no_of_orders      sales
##          0          0          0          0          0
```

```
##Visual Plot of Missing Data
plot_missing(df_clean)
```



```
## Remove Duplicate Entries
# Why: Duplicate rows can skew results (e.g., overstating sales).
# We check the number of rows before and after to see if any were removed.
cat("No of Rows before Removing Duplicates: ", nrow(df_clean), "\n")
```

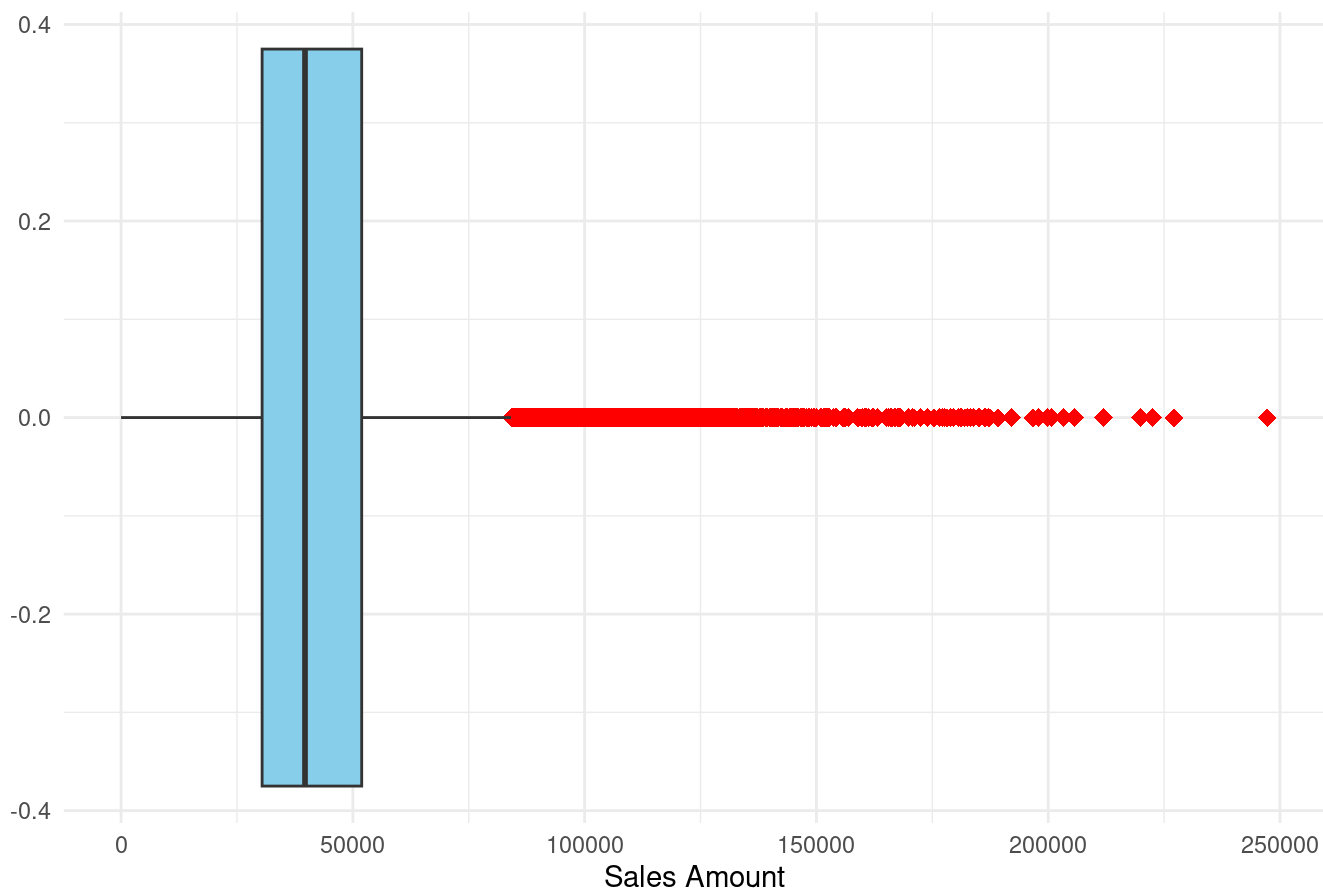
```
## No of Rows before Removing Duplicates: 188340
```

```
duplicated_data=df_clean %>%  
  group_by(across(everything())) %>% #Group by all columns  
  filter(n()>1) %>% # Keep only groups (rows) that appear more than once  
  ungroup()  
  
df_clean=df_clean %>%  
  distinct()  
  
cat("No of Rows after Removing Duplicates: ", nrow(df_clean), "\n")
```

```
## No of Rows after Removing Duplicates: 188340
```

```
## Identify and Handle Outliers  
# Why: Outliers can heavily influence statistical measures (like the mean) and model  
# performance.  
# We first visualize them to understand their distribution.  
  
# Boxplot to visualize potential outliers in sales  
ggplot(df_clean,aes(x=sales)) +  
  geom_boxplot(fill="skyblue",outlier.colour = "red",outlier.shape = 18,outlier.size  
= 3)+  
  labs(title="Boxplot of Sales to Identify Outliers",x="Sales Amount")+  
  theme_minimal()
```

Boxplot of Sales to Identify Outliers



```
# Identify outliers using the Interquartile Range (IQR) method
# Why: The IQR method is a robust statistical way to define outliers.
# An outlier is typically a value that falls below Q1 - 1.5*IQR or above Q3 + 1.5*IQR.

Q1=quantile(df_clean$sales,0.25)
Q3=quantile(df_clean$sales,0.75)
IQR_value=IQR(df_clean$sales)
lower_bound<-Q1-1.5*IQR_value
upper_bound<-Q3+1.5*IQR_value

# Filter out the outliers
# Note: In a real analysis, you might choose to cap them or analyze them separately, in this analysis, we are including them as we are unaware of the reason for outliers.

##Cleaned and Prepared Data frame
final_df <- df_clean
glimpse(final_df)
```

```
## Rows: 188,340
## Columns: 10
## $ id          <chr> "T1000001", "T1000002", "T1000003", "T1000004", "T100000...
## $ store_id    <fct> 1, 253, 252, 251, 250, 249, 248, 247, 246, 254, 245, 11,...
## $ store_type  <fct> S1, S4, S3, S2, S2, S1, S1, S1, S3, S4, S4, S4, S4, S4, ...
## $ location_type <fct> L3, L2, L2, L3, L3, L3, L1, L1, L1, L1, L1, L2, L2, L1, ...
## $ region_code <fct> R1, R1, R1, R1, R4, R2, R2, R3, R3, R1, R2, R1, R2, R1, ...
## $ date        <date> 2018-01-01, 2018-01-01, 2018-01-01, 2018-01-01, 2018-01...
## $ holiday     <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Y...
## $ discount    <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Y...
## $ no_of_orders <dbl> 9, 60, 42, 23, 62, 39, 40, 64, 62, 87, 36, 69, 69, 51, 5...
## $ sales       <dbl> 7011.84, 51789.12, 36868.20, 19715.16, 45614.52, 34211.2...
```

5. Save the Cleaned Data

Why: This saves our cleaned data to a new file. It's a best practice that saves you from re-running the entire cleaning script every time you want to analyze the data.

```
write.csv(final_df, "/home/creation/Project/Sales Analysis Project/sales_data/sales_data_cleaned.csv", row.names = FALSE)
```

6. Calculation, Analysis & Visualization

Get a quick overview of the entire dataset

Why: `summary()` provides key descriptive stats (mean, median, quartiles) for numeric

columns and frequency counts for factor columns.

```
summary(final_df)
```

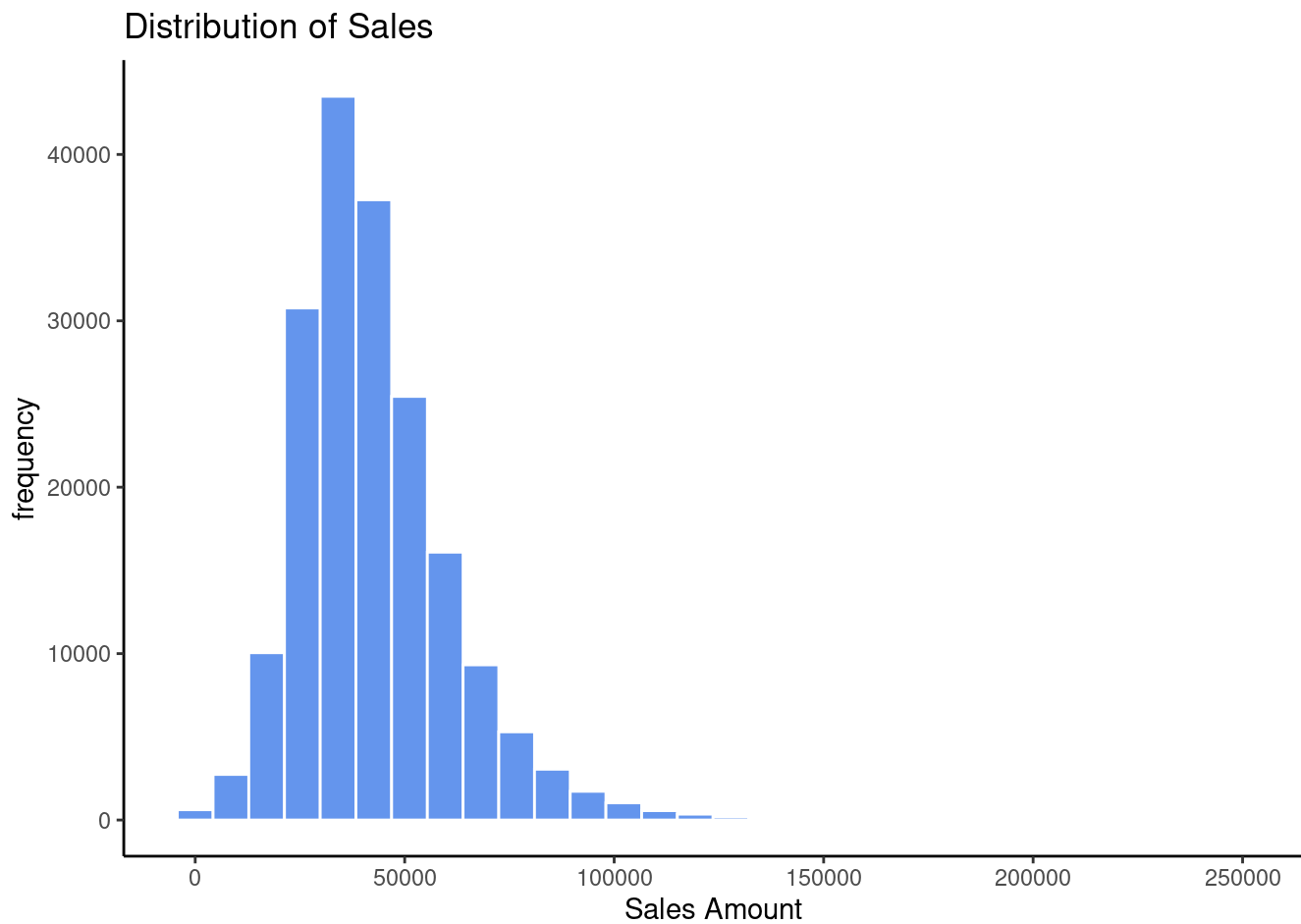


```
##          id          store_id    store_type location_type region_code
## Length:188340      1      :    516    S1:88752    L1:85140      R1:63984
## Class :character    2      :    516    S2:28896    L2:48504      R2:54180
## Mode  :character    3      :    516    S3:24768    L3:29928      R3:44376
##          4      :    516    S4:45924    L4:10836      R4:25800
##          5      :    516
##          6      :    516
##          (Other):185244
##          date          holiday    discount    no_of_orders
## Min.   :2018-01-01    No :163520    No :104051    Min.   : 0.00
## 1st Qu.:2018-05-09    Yes: 24820    Yes: 84289    1st Qu.: 48.00
## Median :2018-09-15
## Mean   :2018-09-15
## 3rd Qu.:2019-01-22
## Max.   :2019-05-31
##
##          sales
## Min.   : 0
## 1st Qu.: 30426
## Median : 39678
## Mean   : 42784
## 3rd Qu.: 51909
## Max.   :247215
##
```

```
## Distribution of Sales Amounts
```

```
# Why: To understand the most common sales values and the shape of the distribution.
```

```
ggplot(final_df,aes(x=sales))+
  geom_histogram(bins=30,fill="cornflowerblue",color="white")+
  labs(title="Distribution of Sales",x="Sales Amount",y="frequency")+
  theme_classic()
```



```
## Store Type Performance
```

```
# Why: To identify which store types generate the most revenue on average.
```

```
final_df %>%
```

```
  group_by(store_type) %>%
```

```
  summarise(
```

```
    average_sales=mean(sales),
```

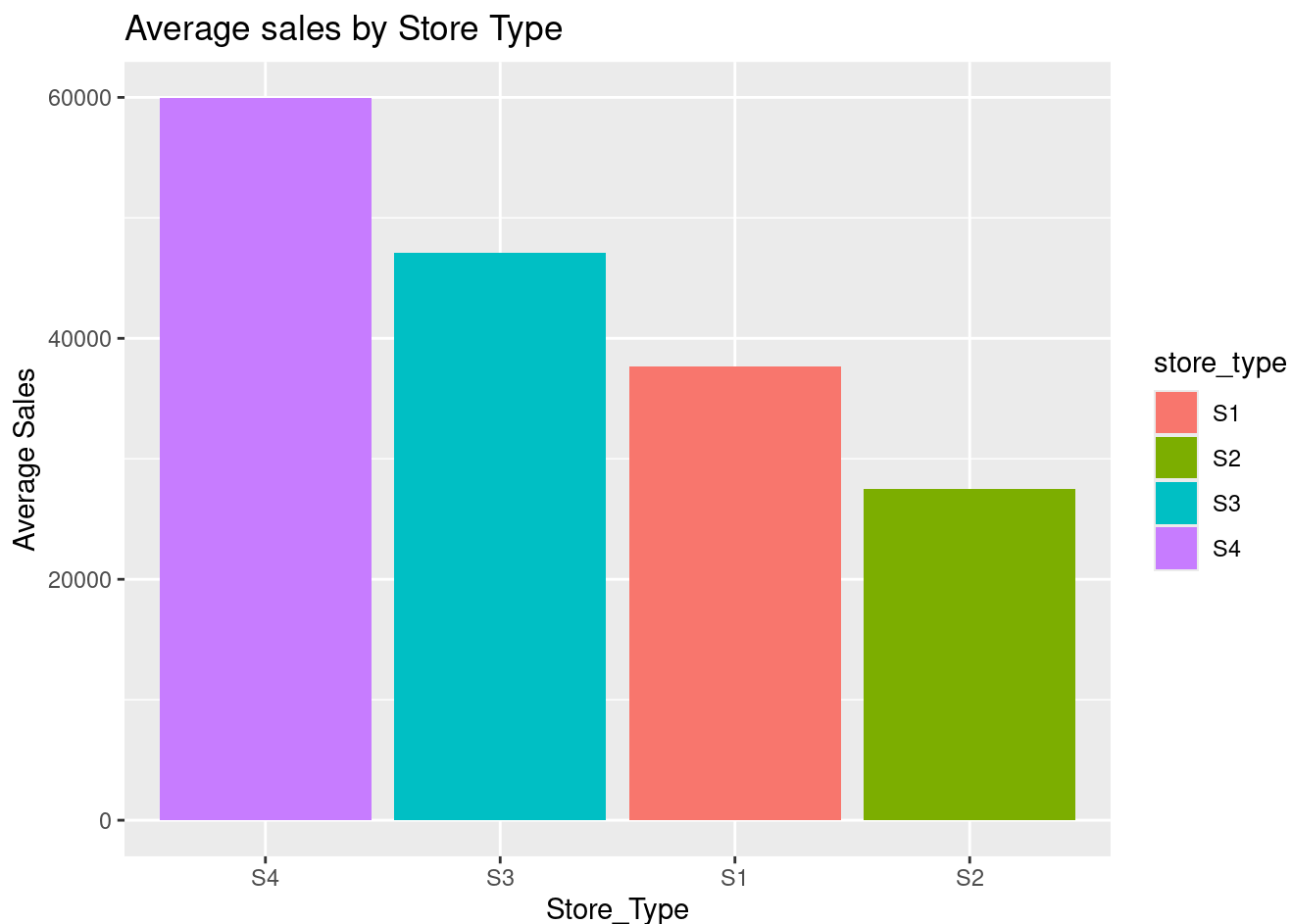
```
    total_sales=sum(sales)
```

```
  ) %>%
```

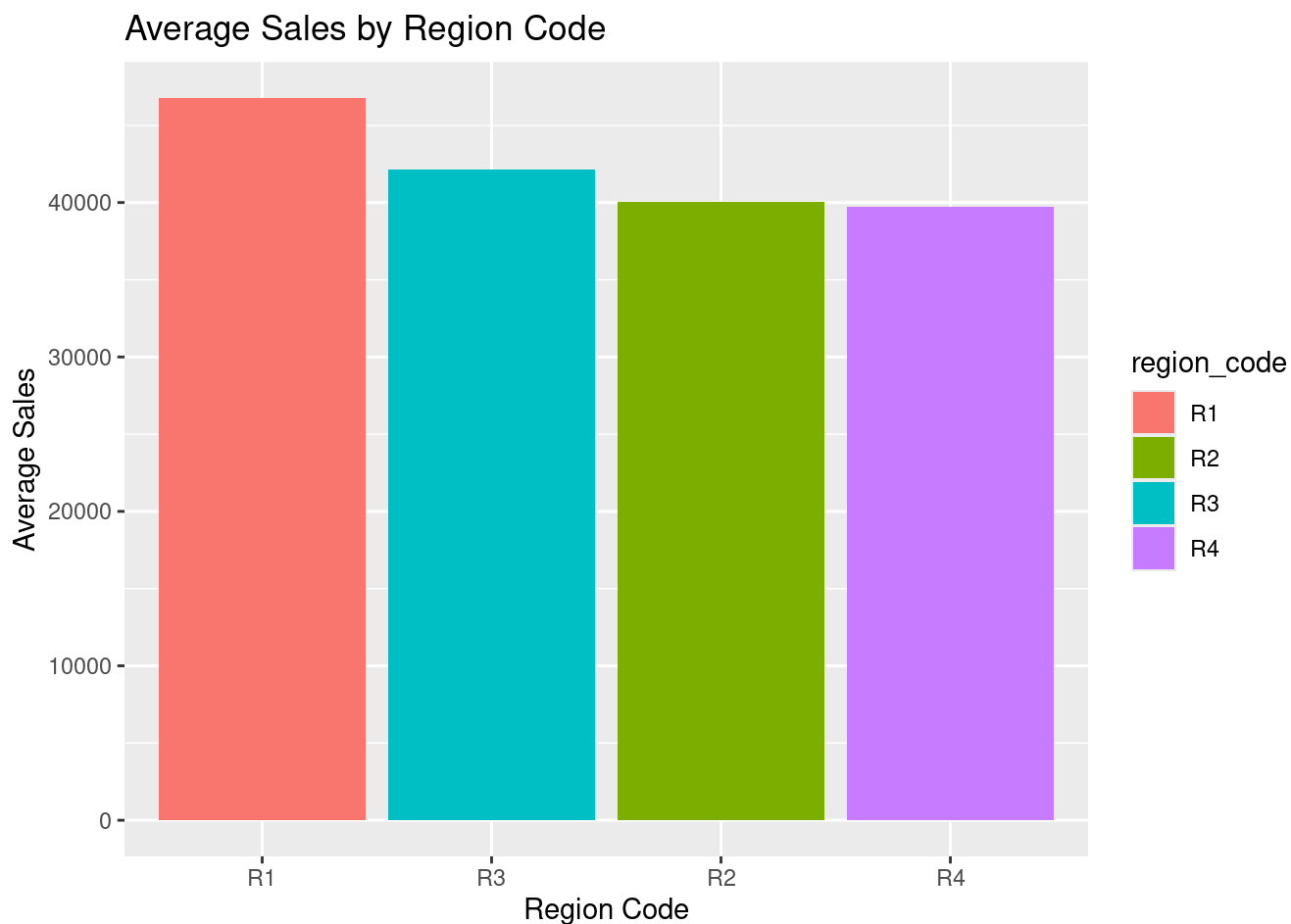
```
  ggplot(aes(x=reorder(store_type, -average_sales), y=average_sales, fill=store_type))+
```

```
  geom_bar(stat = "identity")+
```

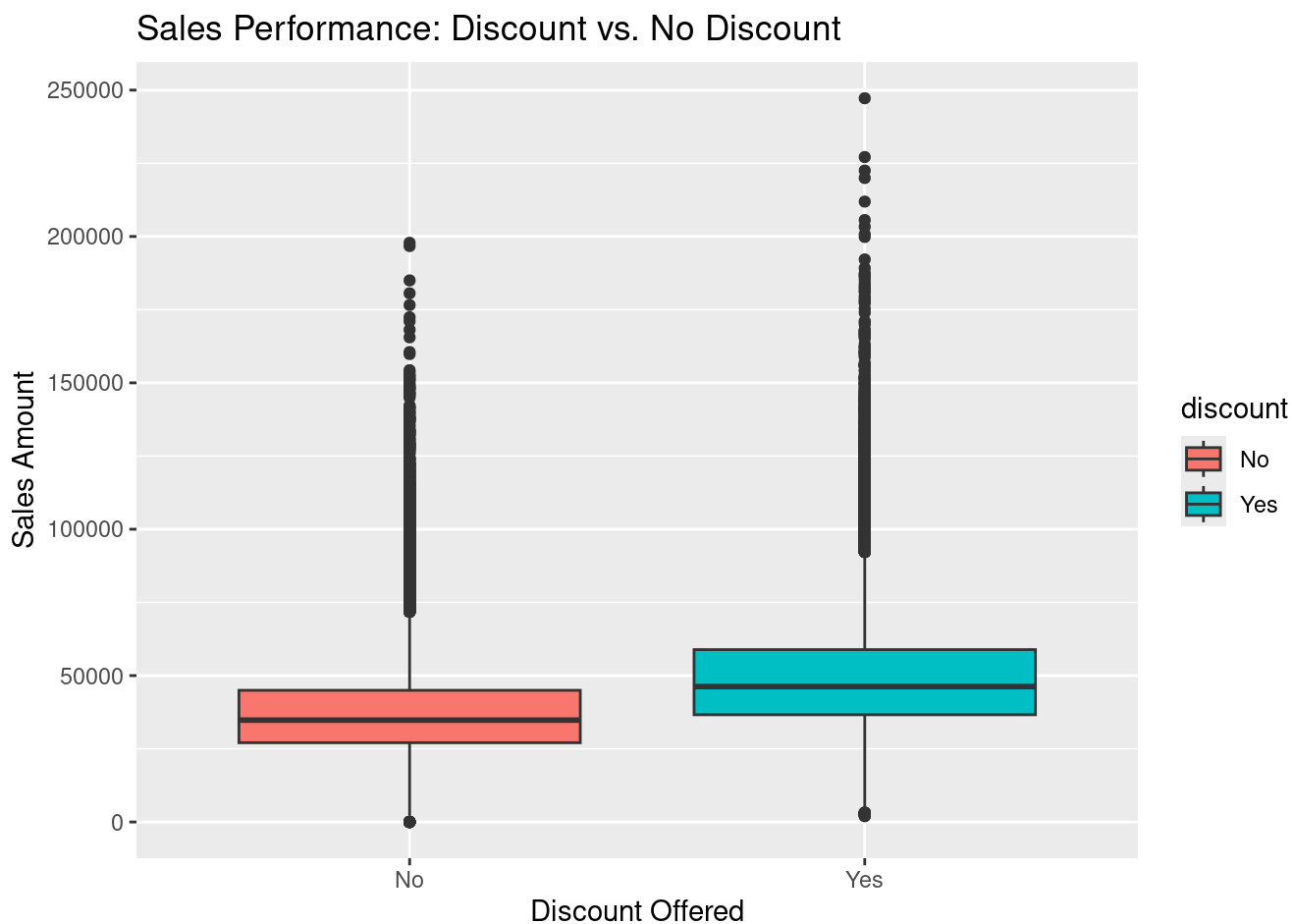
```
  labs(title="Average sales by Store Type", x="Store_Type", y="Average Sales")
```



```
## Regional Performance Analysis
# Why: To see which regions are the most profitable.
final_df %>%
  group_by(region_code) %>%
  summarise(
    average_sales=mean(sales),
    total_sales=sum(sales)
  ) %>%
  ggplot(aes(x=reorder(region_code, -average_sales), y=average_sales, fill=region_cod
e))+
  geom_bar(stat="identity")+
  labs(title="Average Sales by Region Code", x="Region Code", y="Average Sales")
```



```
## Impact of Discounts on Sales
# Why: To visually check if offering a discount corresponds to higher sales.
ggplot(final_df,aes(x=discount,y=sales,fill = discount))+
geom_boxplot()+
labs(title="Sales Performance: Discount vs. No Discount", x = "Discount Offered", y =
"Sales Amount")
```



7. Statistical Analysis

ANOVA to compare sales across store types

Why: A boxplot shows a difference, but ANOVA tells us if that difference is statistically significant.

Null Hypothesis (H_0): The average sales are equal across all store types.

```
anova_result<-aov(sales~store_type,data = final_df)
```

```
summary(anova_result)
```

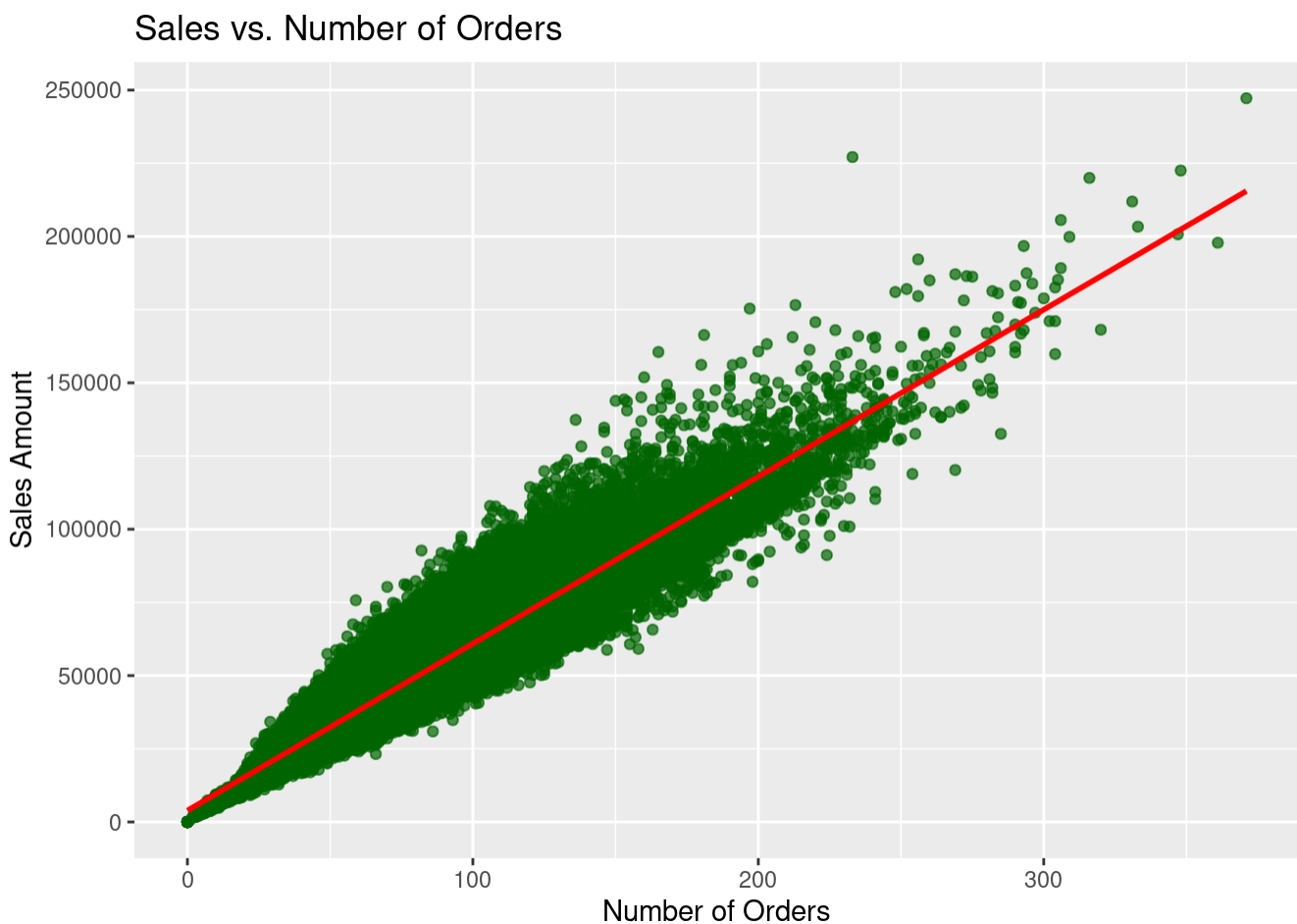
```
##              Df    Sum Sq  Mean Sq F value Pr(>F)
## store_type      3 2.302e+13  7.672e+12   35124 <2e-16 ***
## Residuals 188336 4.114e+13  2.184e+08
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Interpretation: Look at the p-value ( $Pr(>F)$ ). If it's very small (e.g.,  $< 0.05$ ),  
# we reject the null hypothesis and conclude that there is a significant difference  
# in sales between at least two of the store types.  
  
## Correlation between Orders and Sales  
# Why: To quantify the strength and direction of the linear relationship between the  
# number of orders and the total sales amount. We expect a strong positive correlation.  
correlation<-cor(final_df$no_of_orders,final_df$sales)  
cat("Correlation between Orders and Sales:", correlation, "\n")
```

```
## Correlation between Orders and Sales: 0.9416006
```

```
# Visualize this relationship with a scatter plot  
ggplot(final_df, aes(x = no_of_orders, y = sales)) +  
  geom_point(alpha = 0.7, color = "darkgreen") +  
  geom_smooth(method = "lm", color = "red", se = FALSE) + # Adds a linear regression  
  line  
  labs(title = "Sales vs. Number of Orders", x = "Number of Orders", y = "Sales Amount")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
## Hypothesis Testing for Discount Effectiveness (T-test)
# Why: To statistically determine if offering a discount leads to a significant
# increase in average sales.
# Null Hypothesis (H0): The mean sales for discounted and non-discounted days are equal.
t_test_result <- t.test(sales ~ discount, data = final_df)
print(t_test_result)
```

```
##
## Welch Two Sample t-test
##
## data: sales by discount
## t = -145.93, df = 164738, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group No and group Yes is
## not equal to 0
## 95 percent confidence interval:
## -12184.30 -11861.34
## sample estimates:
## mean in group No mean in group Yes
## 37403.68 49426.50
```

Interpretation: Again, look at the p-value. A small p-value (< 0.05) suggests that the observed difference in average sales is not due to random chance, and the discount has a significant effect.

#Advanced Analysis - Sales Prediction Modeling

Multiple Linear Regression

Why: To build a simple model that predicts sales based on multiple factors.

This helps us understand the individual contribution of each factor.

```
regression_model <- lm(sales ~ no_of_orders + store_type + region_code + discount, data = final_df)
summary(regression_model)
```

```
##
## Call:
## lm(formula = sales ~ no_of_orders + store_type + region_code +
##     discount, data = final_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -42477  -3408   -625    2731   84123
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -1339.5716     45.0142  -29.759  < 2e-16 ***
## no_of_orders    601.2975      0.6157  976.577  < 2e-16 ***
## store_typeS2    214.4707     38.7843   5.530 3.21e-08 ***
## store_typeS3    293.7568     40.8634   7.189 6.56e-13 ***
## store_typeS4  -3151.7987     42.6249 -73.943  < 2e-16 ***
## region_codeR2  2519.8096     33.5292  75.153  < 2e-16 ***
## region_codeR3  3963.7181     34.9910 113.278  < 2e-16 ***
## region_codeR4  4492.5930     42.0746 106.777  < 2e-16 ***
## discountYes    3429.3336     26.8653 127.649  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5482 on 188331 degrees of freedom
## Multiple R-squared:  0.9118, Adjusted R-squared:  0.9118
## F-statistic: 2.433e+05 on 8 and 188331 DF,  p-value: < 2.2e-16
```

Interpretation:

- Coefficients: Show the estimated change in sales for a one-unit change in a predictor, holding others constant.

- R-squared: Tells you what percentage of the variation in sales is explained by your model. A higher value is better.

- p-values (Pr(>|t|)): Indicate the significance of each predictor.

8. Machine Learning and Feature Engineering

Step 1: Reverifying and Correcting data types

```
final_df <- final_df %>%
  mutate(
    date = as.Date(date),
    store_id = as.factor(store_id),
    store_type = as.factor(store_type),
    location_type = as.factor(location_type),
    region_code = as.factor(region_code),
    holiday = as.factor(holiday),
    discount = as.factor(discount)
  )
cat("--- Data loaded successfully. Initial structure: ---\n")
```



```
## --- Data loaded successfully. Initial structure: ---
```

```
glimpse(final_df)
```

```
## Rows: 188,340
## Columns: 10
## $ id          <chr> "T1000001", "T1000002", "T1000003", "T1000004", "T100000...
## $ store_id    <fct> 1, 253, 252, 251, 250, 249, 248, 247, 246, 254, 245, 11,...
## $ store_type  <fct> S1, S4, S3, S2, S2, S1, S1, S1, S3, S4, S4, S4, S4, S4, ...
## $ location_type <fct> L3, L2, L2, L3, L3, L3, L1, L1, L1, L1, L1, L2, L2, L1, ...
## $ region_code <fct> R1, R1, R1, R1, R4, R2, R2, R3, R3, R1, R2, R1, R2, R1, ...
## $ date        <date> 2018-01-01, 2018-01-01, 2018-01-01, 2018-01-01, 2018-01...
## $ holiday     <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Y...
## $ discount    <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Y...
## $ no_of_orders <dbl> 9, 60, 42, 23, 62, 39, 40, 64, 62, 87, 36, 69, 69, 51, 5...
## $ sales       <dbl> 7011.84, 51789.12, 36868.20, 19715.16, 45614.52, 34211.2...
```

#Step 2: Efficient Feature Engineering

We are strategically removing 'week_of_year' to prevent the memory explosion.

Its predictive power is mostly captured by the 'month' feature and is not worth the computational cost.

```
df_model_ready <- final_df %>%
  arrange(store_id, date) %>%
  group_by(store_id) %>%
  mutate(
    # Time-based features
    year = as.factor(year(date)),
    month = as.factor(month(date, label = TRUE)),
    day_of_week = as.factor(wday(date, label = TRUE, week_start = 1)),

    # Powerful time-series features
    sales_lag_1 = lag(sales, 1),
    sales_roll_mean_7 = roll_mean(sales, n = 7, align = "right", fill = NA)
  ) %>%
  ungroup() %>%
  na.omit() %>% # Remove rows with NAs created by lag/roll features
  select(
    sales, no_of_orders, store_type, location_type, region_code,
    holiday, discount, year, month, day_of_week,
    sales_lag_1, sales_roll_mean_7
  )

cat("\n--- Data structure after EFFICIENT feature engineering ---\n")
```

```
##
```

```
## --- Data structure after EFFICIENT feature engineering ---
```

```
glimpse(df_model_ready)
```

```
## Rows: 186,150
## Columns: 12
## $ sales          <dbl> 36855.00, 34101.00, 42429.00, 33510.00, 36873.00, 25...
## $ no_of_orders   <dbl> 55, 52, 65, 53, 56, 35, 57, 56, 78, 64, 68, 31, 53, ...
## $ store_type      <fct> S1, S1, S1, S1, S1, S1, S1, S1, S1, S1, S1, S1, S1, ...
## $ location_type   <fct> L3, L3, L3, L3, L3, L3, L3, L3, L3, L3, L3, L3, L3, ...
## $ region_code     <fct> R1, R1, R1, R1, R1, R1, R1, R1, R1, R1, R1, R1, R1, ...
## $ holiday         <fct> No, No, No, No, No, No, No, No, Yes, No, No, No, No, ...
## $ discount        <fct> No, No, No, No, No, No, Yes, Yes, Yes, Yes, Yes, No, ...
## $ year            <fct> 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018...
## $ month           <ord> Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Ja...
## $ day_of_week     <ord> Sun, Mon, Tue, Wed, Thu, Fri, Sat, Sun, Mon, Tue, We...
## $ sales_lag_1     <dbl> 24495.00, 36855.00, 34101.00, 42429.00, 33510.00, 36...
## $ sales_roll_mean_7 <dbl> 36109.83, 39979.71, 39988.29, 37627.29, 36552.43, 33...
```

```
# Step 3: Robust Model Training
# Split the FULL dataset into Training and Testing sets
set.seed(123)
train_indices <- createDataPartition(
  y = df_model_ready$sales,
  p = 0.8,
  list = FALSE
)
train_data <- df_model_ready[train_indices, ]
test_data  <- df_model_ready[-train_indices, ]

# Step 4.1: Strategy 1: Strategic Sampling
# --- STRATEGY 1: STRATEGIC SUBSAMPLING ---
# This is the KEY step to prevent the freeze. We train on a smaller sample.
# 40,000 rows is more than enough to build an excellent model.
set.seed(123)
train_data_sample <- train_data %>% sample_n(40000)

cat("\nOriginal training data rows:", nrow(train_data), "\n")
```

```
##
## Original training data rows: 148921
```

```
cat("Rows in the sample we will use for training:", nrow(train_data_sample), "\n\n")
```

```
## Rows in the sample we will use for training: 40000
```

```
# Step 4.2: Strategy 2: Setup Parallel Processing
# This uses multiple CPU cores on your server to make training much faster.
cores_to_use <- detectCores() - 1 # Leave one core free for system processes
cl <- makePSOCKcluster(cores_to_use)
registerDoParallel(cl)

cat("--- Registered", cores_to_use, "cores for parallel processing. Starting model training. ---\n")
```

```
## --- Registered 3 cores for parallel processing. Starting model training. ---
```

```
# Step 4.3: Define Training Control for Cross-Validation
train_control <- trainControl(
  method = "cv",
  number = 5,
  verboseIter = TRUE, # Shows progress
  allowParallel = TRUE # This tells caret to use our parallel setup
)

#Step 5: Train the XGBoost Model using smaller train data sample
# We use a simplified tuning grid to find good parameters quickly.
cat("\n--- Training and Tuning XGBoost Model ---\n")
```

```
##
## --- Training and Tuning XGBoost Model ---
```

```
xgb_tune_grid_fast <- expand.grid(
  nrounds = c(150, 250),
  max_depth = 6,
  eta = 0.05,
  gamma = 0,
  colsample_bytree = 0.7,
  min_child_weight = 1,
  subsample = 0.7
)

set.seed(123)

# We train on the SAMPLE data to ensure it completes successfully
xgb_tuned_model <- train(
  sales ~ .,
  data = train_data_sample,
  method = "xgbTree",
  trControl = train_control,
  tuneGrid = xgb_tune_grid_fast,
  verbose = FALSE
)
```

```
## Aggregating results
## Selecting tuning parameters
## Fitting nrounds = 250, max_depth = 6, eta = 0.05, gamma = 0, colsample_bytree = 0.7, min_child_weight = 1, subsample = 0.7 on full training set
```

```
cat("\n--- XGBoost Tuning Complete ---\n")
```

```
##
## --- XGBoost Tuning Complete ---
```

```
print(xgb_tuned_model)
```

```
## eXtreme Gradient Boosting
##
## 40000 samples
##    11 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 32000, 32002, 31999, 32000, 31999
## Resampling results across tuning parameters:
##
##  nrounds  RMSE      Rsquared   MAE
##   150      3412.397  0.9652124  2424.514
##   250      3271.470  0.9679520  2318.755
##
## Tuning parameter 'max_depth' was held constant at a value of 6
## Tuning
## parameter 'min_child_weight' was held constant at a value of 1
##
## Tuning parameter 'subsample' was held constant at a value of 0.7
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were nrounds = 250, max_depth = 6, eta
## = 0.05, gamma = 0, colsample_bytree = 0.7, min_child_weight = 1 and
## subsample = 0.7.
```

```
# --- IMPORTANT: Stop the Parallel Cluster ---
# Always do this after training is complete to release the CPU cores.
stopCluster(cl)
registerDoSEQ() # Unregister the parallel backend
cat("\n--- Parallel cluster stopped. ---\n")
```

```
##
## --- Parallel cluster stopped. ---
```

```
#Step 6: Final Model Evaluation
# We evaluate the trained model on the FULL, unseen test set to get a true measure of
performance.
cat("\n--- Final Model Evaluation on the Full, Unseen Test Set ---\n")
```

```
##
## --- Final Model Evaluation on the Full, Unseen Test Set ---
```

```
predictions_xgb <- predict(xgb_tuned_model, newdata = test_data)

# Calculate performance metrics
performance_metrics <- data.frame(
  RMSE = RMSE(predictions_xgb, test_data$sales),
  MAE = MAE(predictions_xgb, test_data$sales),
  Rsquared = R2(predictions_xgb, test_data$sales)
)
print(round(performance_metrics, 4))
```

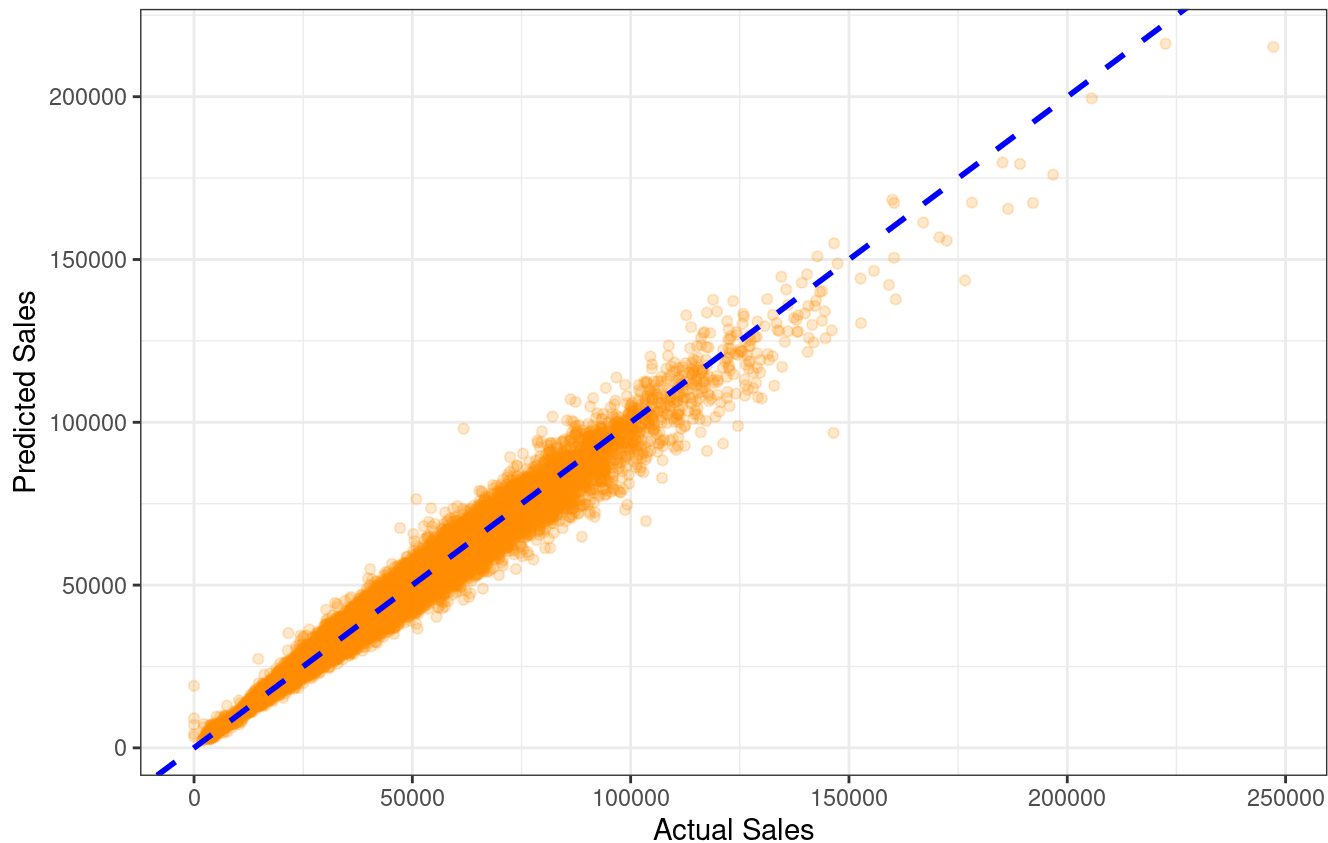
```
##          RMSE          MAE Rsquared
## 1 3257.582 2306.89    0.9691
```

```
# Visualize Actual vs. Predicted values for our model
results_df <- data.frame(actual = test_data$sales, predicted = predictions_xgb)

ggplot(results_df, aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.2, color = "darkorange") +
  geom_abline(color = "blue", linetype = "dashed", linewidth = 1) +
  labs(
    title = "Actual vs. Predicted Sales (Tuned XGBoost Model)",
    subtitle = paste("Test Set R-squared:", round(performance_metrics$Rsquared, 3)),
    x = "Actual Sales",
    y = "Predicted Sales"
  ) +
  theme_bw()
```

Actual vs. Predicted Sales (Tuned XGBoost Model)

Test Set R-squared: 0.969



```
#Step 7 What if Scenarios
```

```
# Now we use our best model (xgb_tuned_model) as a business simulator.
```

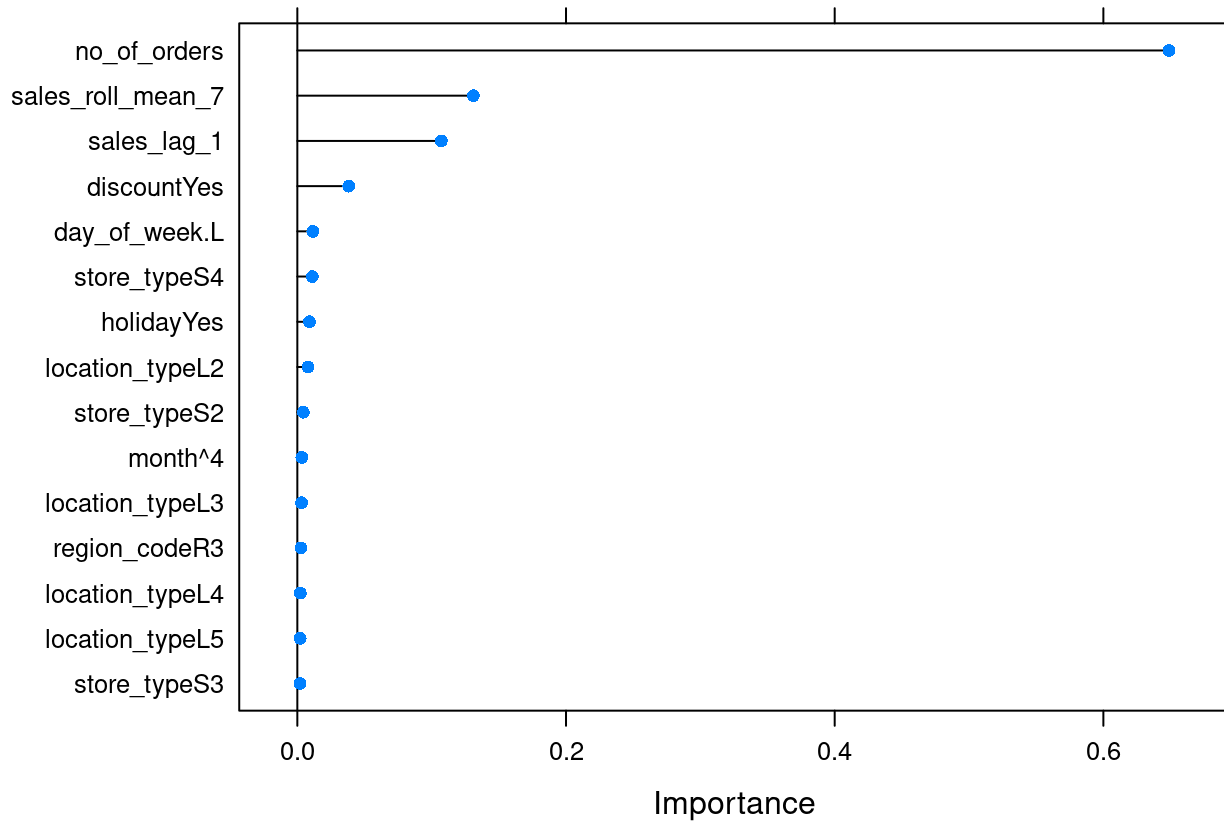
```
# Analyze Feature Importance
```

```
# This tells us WHICH variables are the most powerful drivers of sales.
```

```
importance <- varImp(xgb_tuned_model, scale = FALSE)
```

```
plot(importance, top = 15, main = "Top 15 Most Important Features (XGBoost)")
```

Top 15 Most Important Features (XGBoost)



```
# "What-If" Scenario 1: What is the impact of increasing order count?
# Define a typical scenario. We need to provide values for ALL features in the model.
base_scenario <- list(
  no_of_orders = 70, # This is the variable we will change
  store_type = "S1",
  location_type = "L1",
  region_code = "R1",
  holiday = "No",
  discount = "Yes",
  year = "2018", # Use a year present in your data
  month = "Jul",
  day_of_week = "Mon",
  sales_lag_1 = 45000,
  sales_roll_mean_7 = 48000
)

# Create a sequence of order counts to test
order_counts_to_test <- seq(from = 40, to = 150, by = 5)
predicted_sales_list <- c()

for (orders in order_counts_to_test) {
  current_scenario <- base_scenario
  current_scenario$no_of_orders <- orders

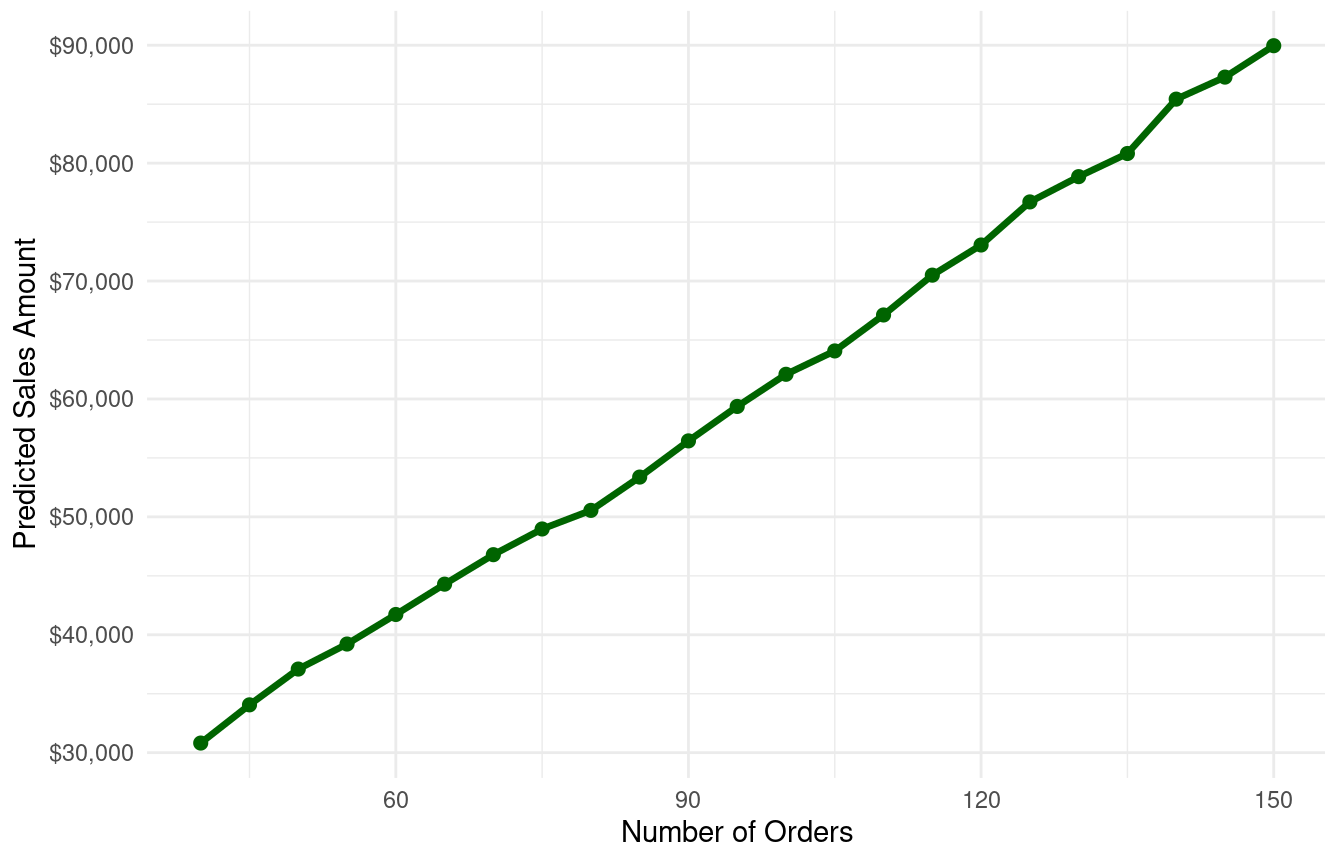
  # The `predict` function from caret handles all the data formatting for us!
  predicted_value <- predict(xgb_tuned_model, newdata = as.data.frame(current_scenario))
  predicted_sales_list <- c(predicted_sales_list, predicted_value)
}

scenario1_results <- data.frame(orders = order_counts_to_test, predicted_sales = predicted_sales_list)

# Plot the results of the simulation
ggplot(scenario1_results, aes(x = orders, y = predicted_sales)) +
  geom_line(color = "darkgreen", linewidth = 1.2) +
  geom_point(color = "darkgreen", size = 2) +
  labs(
    title = "What-if Analysis: Impact of Order Count on Sales",
    subtitle = "Holding all other factors constant for a sample store",
    x = "Number of Orders",
    y = "Predicted Sales Amount"
  ) +
  scale_y_continuous(labels = scales::dollar_format()) +
  theme_minimal()
```


What-if Analysis: Impact of Order Count on Sales

Holding all other factors constant for a sample store



```
# "What-If" Scenario 2: How effective are discounts across different store types? ---
# We use `expand.grid` to create every combination of store_type and discount to test
scenario2_df <- expand.grid(
  store_type = c("S1", "S2", "S3", "S4"), # Test all store types in your data
  discount = c("No", "Yes")
)

# Add the other constant information to our scenarios dataframe
scenario2_df <- scenario2_df %>%
  mutate(
    no_of_orders = 80, # Assume a fixed 80 orders for a fair comparison
    location_type = "L1",
    region_code = "R1",
    holiday = "No",
    year = "2018",
    month = "Jul",
    day_of_week = "Mon",
    sales_lag_1 = 45000,
    sales_roll_mean_7 = 48000
  )

# Get a prediction for each scenario (each row in the dataframe)
scenario2_df$predicted_sales <- predict(xgb_tuned_model, newdata = scenario2_df)

# Plot the results as a grouped bar chart for easy comparison
ggplot(scenario2_df, aes(x = store_type, y = predicted_sales, fill = discount)) +
  geom_bar(stat = "identity", position = "dodge") +
  geom_text(
    aes(label = scales::dollar(predicted_sales, accuracy = 1)),
    position = position_dodge(width = 0.9), vjust = -0.5, size = 3.5
  ) +
  labs(
    title = "What-if: Discount Effectiveness Across Store Types",
    subtitle = "Assuming a fixed 80 orders per day",
    x = "Store Type",
    y = "Predicted Sales Amount",
    fill = "Discount Offered?"
  ) +
  scale_fill_manual(values = c("No" = "#E57373", "Yes" = "#64B5F6")) +
  theme_minimal()
```

What-if: Discount Effectiveness Across Store Types

Assuming a fixed 80 orders per day

